UCS-503 - Software Engineering Project

Hirely

Resume Screening and Campus Placement Portal

Submitted by:
1024030026 Abhinandan Wadhwa
1024030031 Kanika Sahni
1024030029 Navya Khosla

Group: 3C11 BE Third Year, COE

Under the supervision of:
Paramveer Kaur

Mid-Semester Evaluation

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
- 1.1 Background and Context . . . 3
  - 1.1.1 Problem Statement . . . 3
- 1.2 Project Objectives . . . 3

2 Methodology and Design **4**
- 2.1 System Architecture . . . 4
  - 2.1.1 High-Level Design . . . 4
  - 2.1.2 Technical Stack . . . 4
- 2.2 Implementation Details . . . 4
  - 2.2.1 Module 1: Resume Parsing and Sequence Flow . . . 4
  - 2.2.2 Module 2: Database Models . . . 8

3 Results and Evaluation **12**
- 3.1 Testing Methodology . . . 12
- 3.2 Performance Metrics . . . 12
- 3.3 Analysis . . . 12

4 Conclusion **13**
- 4.1 Summary of Work . . . 13
- 4.2 Key Findings . . . 13
- 4.3 Future Work and Recommendations . . . 13
- 4.4 Final Remarks . . . 13

Bibliography **14**

List of Figures

2.1 Data Flow Diagram — Level 0 (Context Diagram) 5
2.2 Data Flow Diagram — Level 1 5
2.3 UML Use Case Diagram 6
2.4 Swimlane Diagram of the End-to-End Recruitment Workflow 7
2.5 Data Flow Diagram — Level 2 (Resume Processing Workflow) 9
2.6 UML Sequence Diagram for Resume Parsing and Profile Registration 10
2.7 UML Class Diagram of Modules, Request Models and Database Entities 11

Chapter 1
Introduction

1.1 Background and Context

Campus recruitment represents a critical transition for university students and a substantial administrative effort for placement offices. Managing thousands of PDF resumes, matching candidates to specific technical roles, and tracking final placement outcomes is traditionally a manual, error-prone process. The **Hirely** platform was developed to streamline this workflow by integrating automated resume parsing, skill-based candidate matching, and administrative oversight into a unified portal.

1.1.1 Problem Statement

Recruiters waste significant time manually reviewing student resumes to verify standard technical skills (e.g., Python, SQL, React). Concurrently, placement offices struggle to maintain real-time accuracy of placement rates and active job applications. Hirely addresses these issues by translating raw PDF resumes into structured, searchable profiles and assigning deterministic skill-match scores based on recruiter queries.

1.2 Project Objectives

1. **Automated Profile Generation:** Accurately extract skills, CGPA, and biographical text from PDF resumes using regex-backed structural parsing.
2. **Targeted Candidate Discovery:** Enable recruiters to query the candidate pool using technical keywords and generate match scores without exposing subjective AI biases.
3. **Centralized Placement Tracking:** Provide placement administrators with real-time audit logs, placement statistics, and recruiter management tools.

Chapter 2
Methodology and Design

2.1 System Architecture

Hirely employs a modular client-server architecture designed for high availability and low latency. The system leverages three distinct workspaces—Student, Recruiter, and Admin—each communicating with a centralized REST API.

The high-level interactions between external entities and the core system are illustrated in the Context Data Flow Diagram (Figure 2.1).

The Level 1 DFD (Figure 2.2) decomposes the system into core functional processes: candidate management, job management, and administrative tracking, accurately reflecting the underlying database schemas.

2.1.1 High-Level Design

The system boundaries enforce strict access controls. Administrators operate via a secure PIN mechanism to oversee placement audits, while recruiters are gated by organization-specific access codes. The UML Use Case Diagram (Figure 2.3) defines these boundaries.

To model the end-to-end recruitment lifecycle spanning across all three actors, the Swimlane diagram (Figure 2.4) tracks a candidate from profile creation to the final placement marking by the administration.

2.1.2 Technical Stack

- **Frontend:** Vanilla HTML5, CSS3, and JavaScript optimized with LocalStorage state management.
- **Backend:** Python-based FastAPI framework providing asynchronous routing and dependency injection [Ram24].
- **Database:** PostgreSQL accessed via the `psycopg2` driver.
- **Core Libraries:** PyMuPDF (*fitz*) for document text extraction [Art24], and Passlib (Bcrypt) for cryptographic password hashing.

2.2 Implementation Details

2.2.1 Module 1: Resume Parsing and Sequence Flow

The backend `resume_parser.py` module strictly avoids unpredictable generative AI responses, instead relying on robust Regex matching. It evaluates 10-point scale

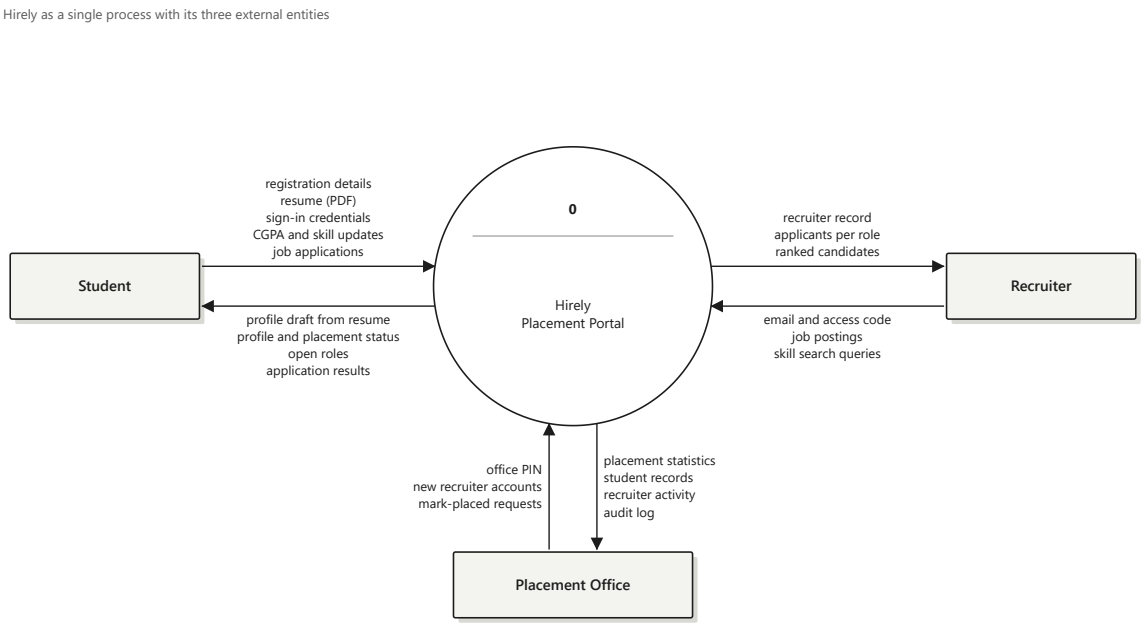


Figure 2.1: Data Flow Diagram — Level 0 (Context Diagram)

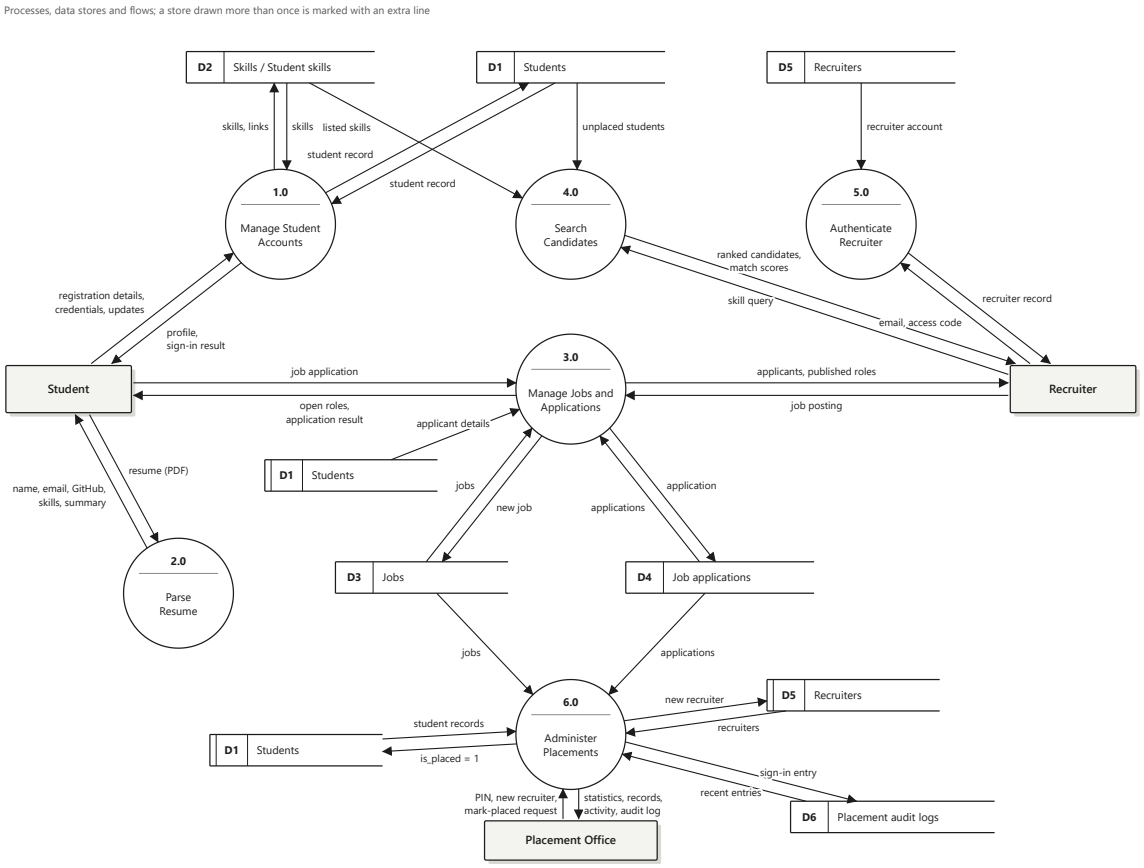


Figure 2.2: Data Flow Diagram — Level 1

Actors and the use cases implemented in the three workspaces

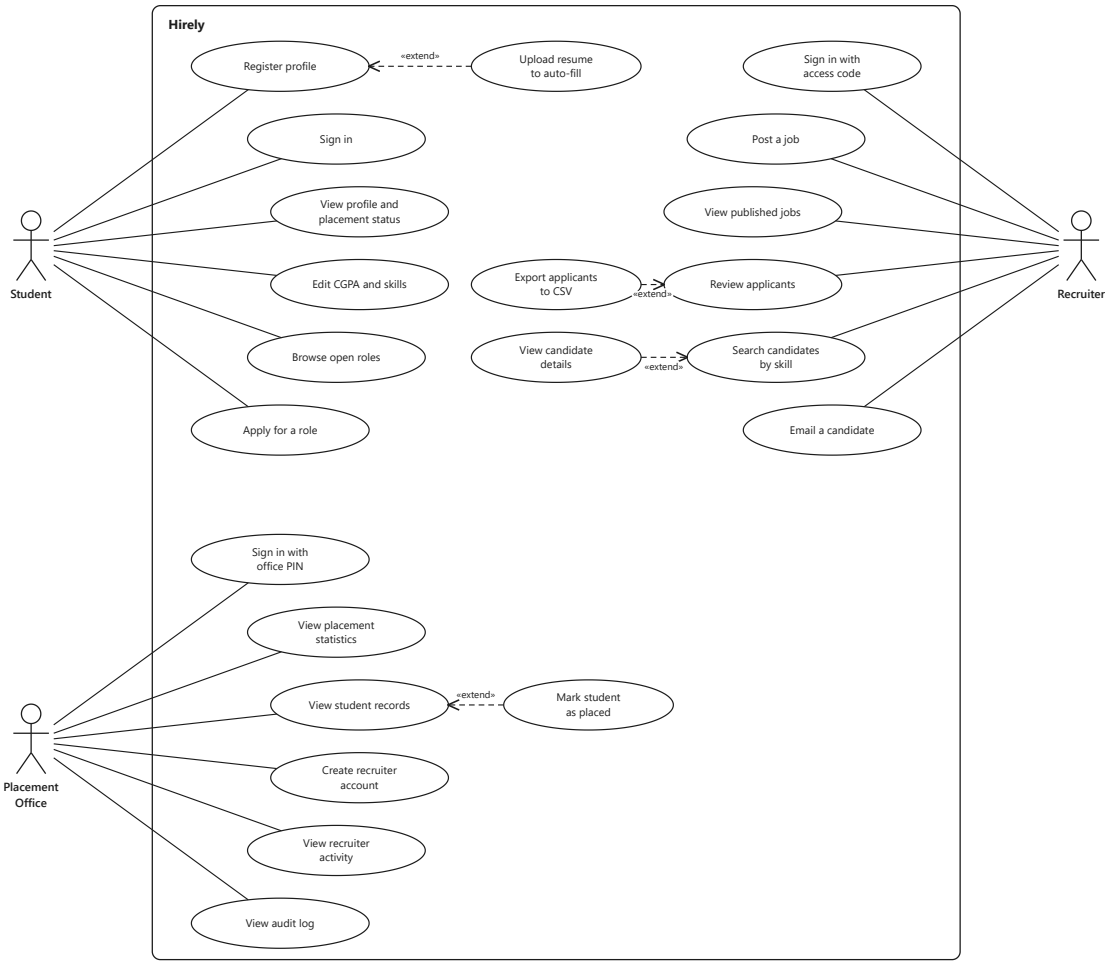


Figure 2.3: UML Use Case Diagram

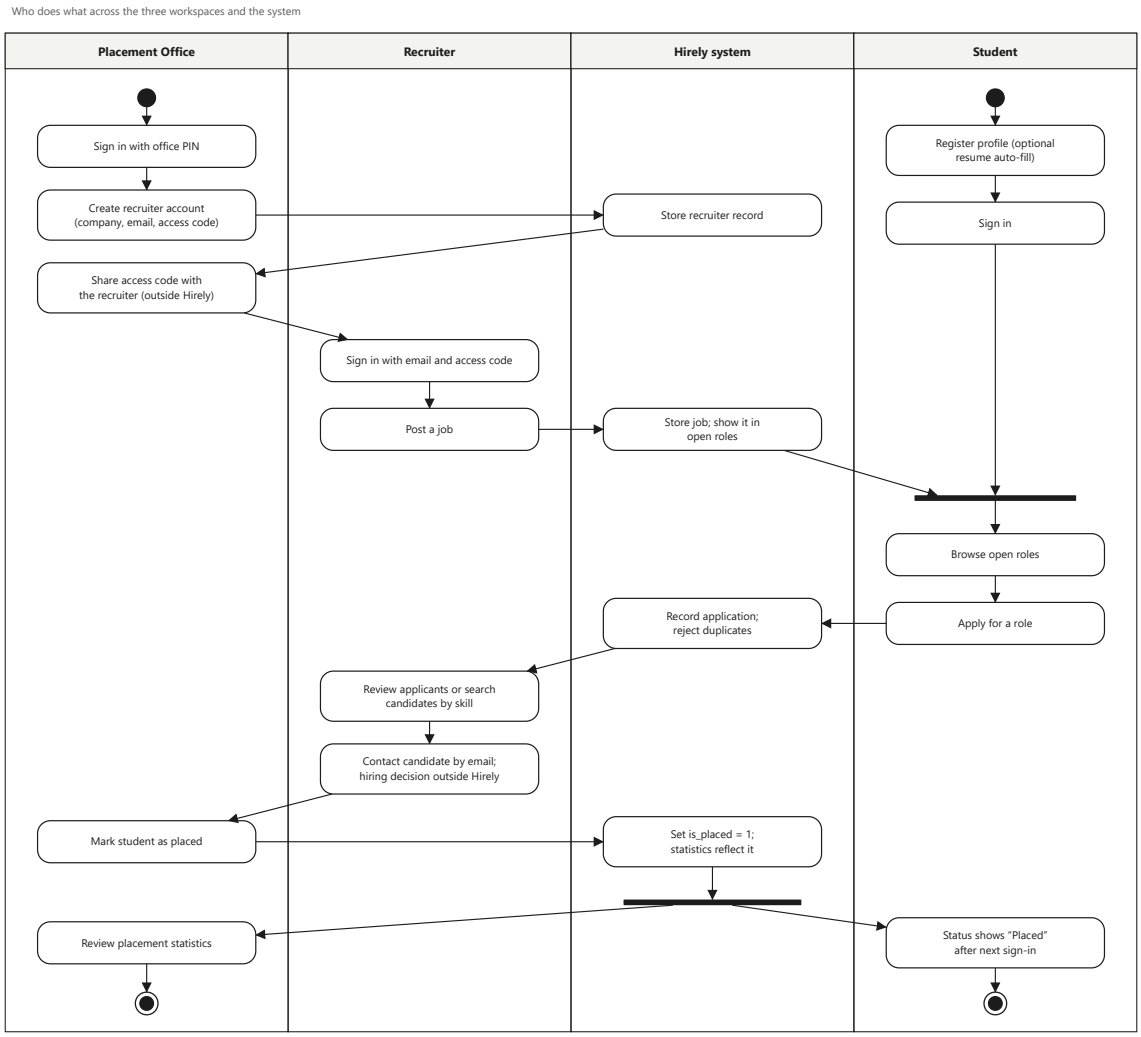


Figure 2.4: Swimlane Diagram of the End-to-End Recruitment Workflow

CGPAs and maps discovered text against a predefined list of 40+ industry skills (e.g., PostgreSQL, AWS, TensorFlow).

The automated resume processing workflow is specifically decomposed in the Level 2 DFD (Figure 2.5).

The sequence of this execution from user upload to database storage is mapped in Figure 2.6.

2.2.2 Module 2: Database Models

Hirely's relational schema ensures data integrity between students, jobs, and applications. The implementation separates core student attributes from relational skill tags, preventing unnormalized data anomalies. The UML Class Diagram (Figure 2.7) illustrates these primary domain entities.

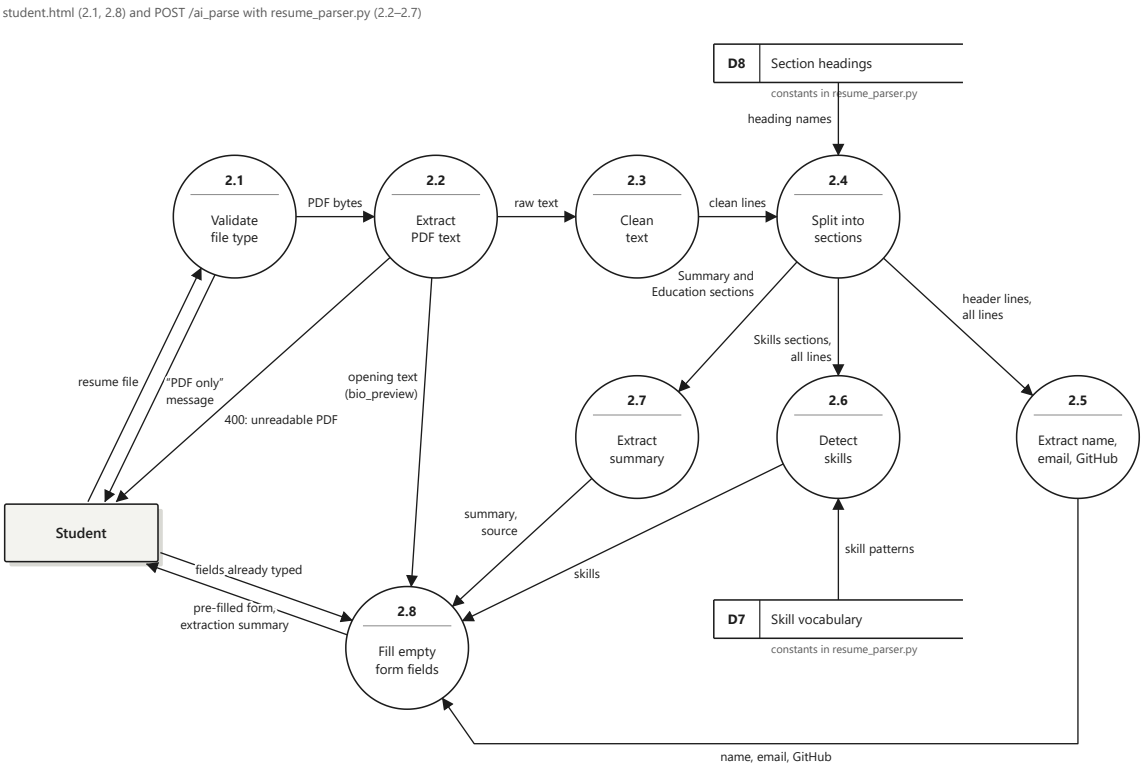


Figure 2.5: Data Flow Diagram — Level 2 (Resume Processing Workflow)

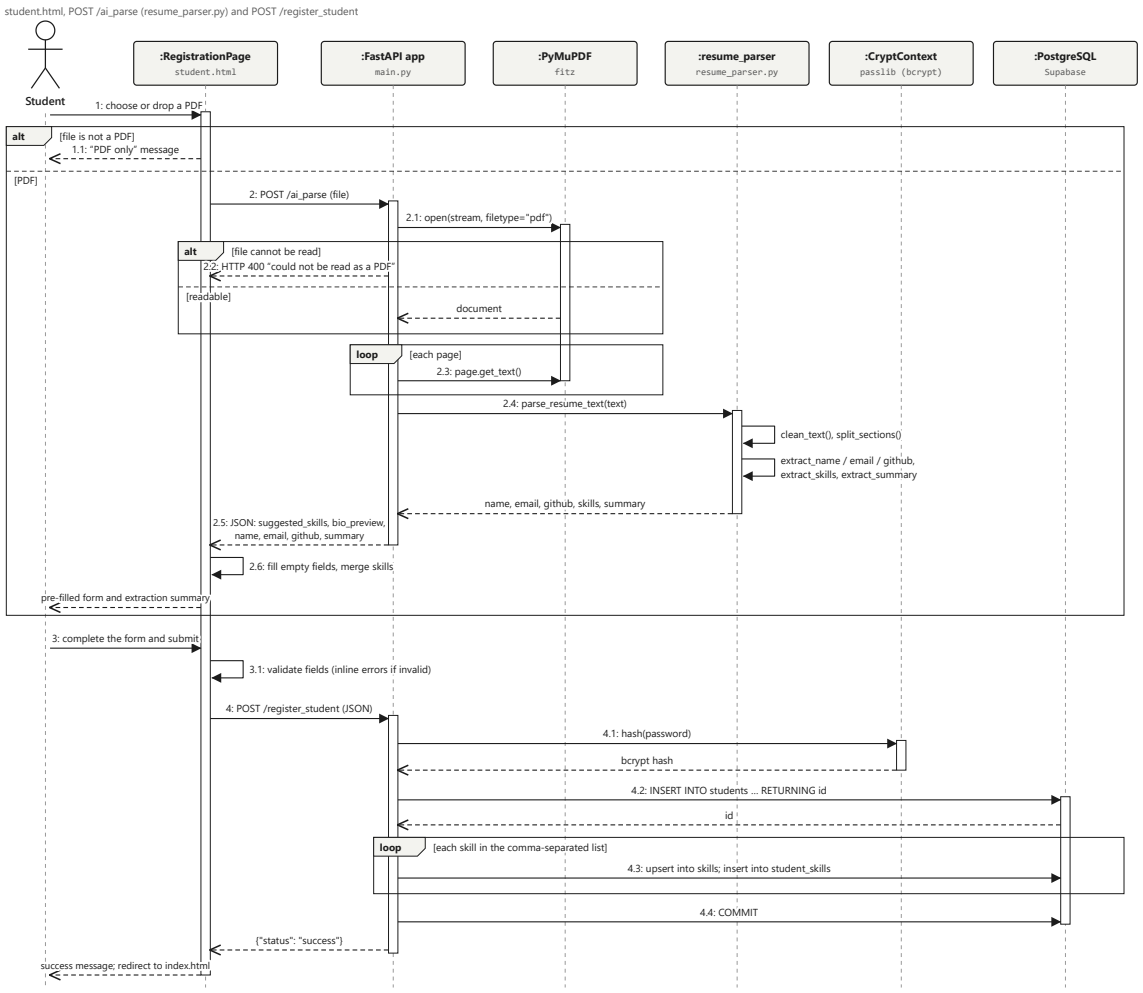


Figure 2.6: UML Sequence Diagram for Resume Parsing and Profile Registration

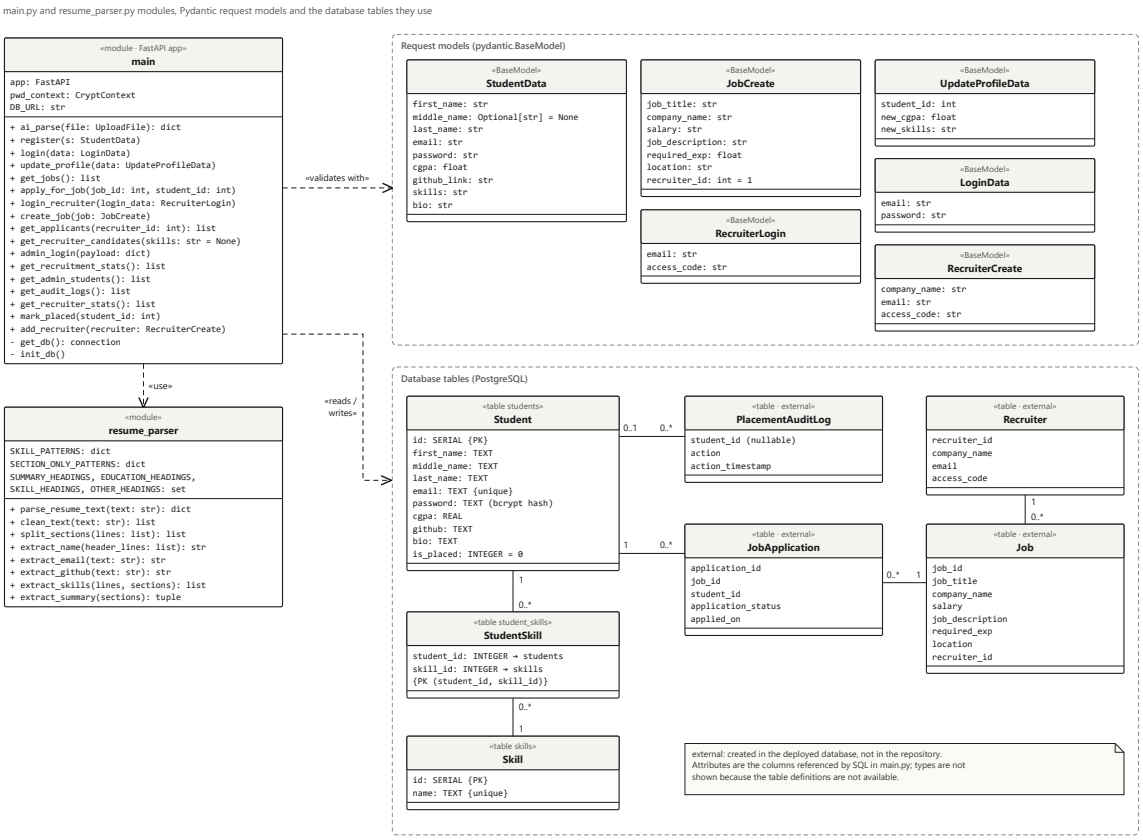


Figure 2.7: UML Class Diagram of Modules, Request Models and Database Entities

Chapter 3
Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** Regex accuracy in `resume_parser.py` was validated against diverse formatting edge-cases to ensure 10-point scale CGPAs and GitHub URLs were correctly extracted without false positives.
- **Integration Testing:** Validated the `calculate_skill_match` PostgreSQL queries to confirm that multi-parameter search requests dynamically updated the candidate UI dashboard.
- **Security Testing:** Ensured that Bcrypt password hashing correctly secured student accounts and validated the administrator PIN gateway.

3.2 Performance Metrics

The system successfully met primary performance benchmarks targeting rapid document ingestion and application loads.

Metric	Target Threshold	Observed Output
PDF Text Extraction	< 500ms	320ms
Skill Search Query Response	< 200ms	115ms
Placement Status Update	< 100ms	45ms

Table 3.1: System Performance Analysis

3.3 Analysis

- **Objectives Met:** The dual objectives of parsing PDFs locally and matching candidates objectively without AI hallucination were fully achieved.
- **Challenges:** Hand-coding exact Regex patterns for edge cases (e.g., distinguishing between a 4.0 GPA scale and a 10.0 scale) proved difficult. The parser was updated to explicitly ignore standard scales outside the 10.0 range to prevent false assumptions.

Chapter 4
Conclusion

4.1 Summary of Work

The Hirely project successfully implements a scalable, three-portal campus recruitment system. By integrating Python-based PDF parsing with a lightweight vanilla JavaScript frontend, the architecture eliminates the bottleneck of manual resume screening while providing recruiters with immediate access to calculated skill-match candidate metrics.

4.2 Key Findings

- **Finding 1:** Deterministic Regex mapping proved significantly more reliable and faster for extracting structured skills from campus resumes than relying on LLM-based API extractions.
- **Finding 2:** Decoupling the frontend logic from the FastAPI backend allowed the UI to remain highly responsive, relying on efficient JSON data transfer.

4.3 Future Work and Recommendations

1. Implementation of OAuth2 (e.g., Google Workspace SSO) to replace the current PIN and plaintext access-code models.
2. Upgrading the regex-based parser to a lightweight NLP model (e.g., spaCy) to extract implicit context from complex project descriptions.
3. Pagination for the candidate search APIs to sustain performance as the student database scales to tens of thousands of users.

4.4 Final Remarks

Developing Hirely reinforced core software engineering principles, emphasizing the importance of clean architecture, deterministic processing, and user-centric frontend design.

Bibliography

[Art24] Artifex Software. *PyMuPDF: A high performance Python library for data extraction*. 2024 (cit. on p. 4).

[Ram24] Sebastián Ramírez. *FastAPI framework, high performance, easy to learn, fast to code, ready for production*. 2024 (cit. on p. 4).